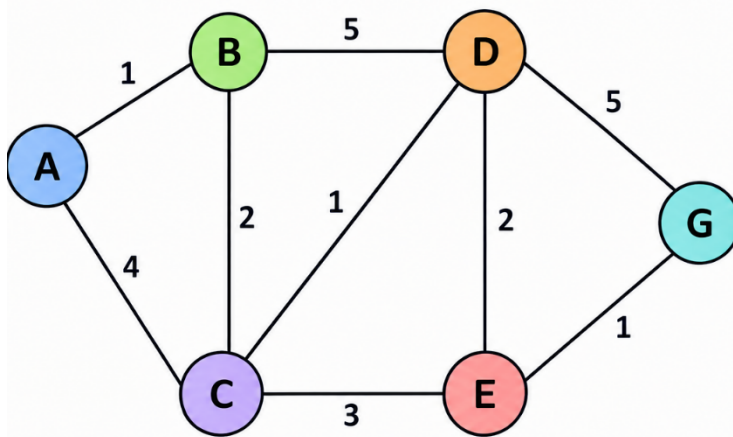


OPRACTICAL-4

Write a program to implement heuristic search algorithms (greedy best-first search, A* search) using PYTHON

Graph Representation

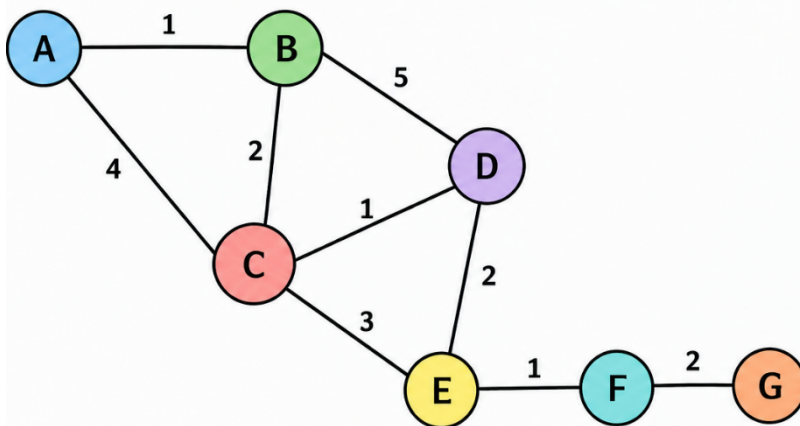


Adjacency List (Graph)

A : { 'B': 1, 'C': 4 }
B : { 'A': 1, 'C': 2, 'D': 5 }
C : { 'A': 4, 'B': 2, 'D': 1, 'E': 3 }
D : { 'B': 5, 'C': 1, 'E': 2, 'G': 5 }
E : { 'C': 3, 'D': 2, 'G': 1 }
G : { 'D': 5, 'E': 1 }

(Edges are undirected with weights as given in the dictionary)

Graph Representation



Adjacency List (as given)

A: {B: 1, C: 4}
B: {A: 1, C: 2, D: 5}
C: {A: 4, B: 2, D: 1, E: 3}
D: {B: 5, C: 1, E: 2}
E: {C: 3, D: 2, F: 1}
F: {E: 1, G: 2}
G: {F: 2}

Legend

(X) = Node
5 = Edge with weight

```

import heapq

# Graph with heuristic values
graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'A': 1, 'C': 2, 'D': 5},
    'C': {'A': 4, 'B': 2, 'D': 1, 'E': 3},
    'D': {'B': 5, 'C': 1, 'E': 2},
    'E': {'C': 3, 'D': 2, 'F': 1},
    'F': {'E': 1, 'G': 2},
    'G': {'F': 2}
}

# Heuristic values: estimated distance to goal G
heuristic = {
    'A': 10,
    'B': 8,
    'C': 6,
    'D': 4,
    'E': 3,
    'F': 2,
    'G': 0
}

def greedy_best_first_search(graph, start, goal, heuristic):

    # Priority queue: (heuristic value, node)
    priority_queue = [(heuristic[start], start)]

    visited = set()
    parent = {start: None}

    while priority_queue:

```

```

h, current = heapq.heappop(priority_queue)

if current in visited:
    continue

visited.add(current)

print("Visiting:", current)

# Goal reached
if current == goal:
    break

# Add unvisited neighbors
for neighbor in graph[current]:

    if neighbor not in visited:
        heapq.heappush(
            priority_queue,
            (heuristic[neighbor], neighbor)
        )

    if neighbor not in parent:
        parent[neighbor] = current

# Check whether goal was reached
if goal not in visited:
    return None

# Construct path

```

```
path = []  
current = goal
```

```
while current is not None:  
    path.append(current)  
    current = parent[current]
```

```
path.reverse()
```

```
return path
```

```
# Starting and goal nodes
```

```
start = 'A'
```

```
goal = 'G'
```

```
# Run Greedy Best-First Search
```

```
path = greedy_best_first_search(  
    graph,  
    start,  
    goal,  
    heuristic  
)
```

```
print("\nGreedy Best-First Search Path:")
```

```
print(" -> ".join(path))
```

```
# -----
```

```
# A* Search
```

```
# -----
```

```
def a_star_search(graph, heuristic, start, goal):
```

```
    priority_queue = []
```

```
    # (f(n), g(n), current node, path)
```

```
    heapq.heappush(priority_queue, (heuristic[start], 0, start, [start]))
```

```
    best_cost = {start: 0}
```

```
    while priority_queue:
```

```
        f, g, current, path = heapq.heappop(priority_queue)
```

```
        print("Visiting:", current, "g =", g, "h =", heuristic[current], "f =", f)
```

```
        # Goal test
```

```
        if current == goal:
```

```
            return path, g
```

```
        # Explore neighbors
```

```
        for neighbor, cost in graph[current].items():
```

```
            new_g = g + cost
```

```
            # If a cheaper path is found
```

```
            if neighbor not in best_cost or new_g < best_cost[neighbor]:
```

```
                best_cost[neighbor] = new_g
```

```

#  $f(n) = g(n) + h(n)$ 
new_f = new_g + heuristic[neighbor]

heapq.heappush(
    priority_queue,
    (new_f, new_g, neighbor, path + [neighbor])
)

```

```

return None, None

```

```

# -----
# Main Program
# -----

```

```

start = 'A'
goal = 'G'

```

```

print("\n===== GREEDY BEST-FIRST SEARCH =====")

```

```

greedy_path = greedy_best_first_search(
    graph, heuristic, start, goal
)

```

```

print("Greedy Best-First Path:", " -> ".join(greedy_path))

```

```

print("\n===== A* SEARCH =====")

```

```

a_star_path, a_star_cost = a_star_search(
    graph, heuristic, start, goal
)

```

```
)
```

```
print("A* Path:", " -> ".join(a_star_path))
```

```
print("A* Path Cost:", a_star_cost)
```

```
*****
```

Write a program to implement heuristic search algorithms (greedy best-first search, A* search) using matlab

```
*****
```

```
clc;
```

```
clear;
```

```
close all;
```

```
% -----
```

```
% Graph representation using adjacency matrix
```

```
% -----
```

```
% Nodes: 1, 2, 3, 4, 5, 6
```

```
%
```

```
% 1
```

```
% /\
```

```
% 2 3
```

```
% /\ \
```

```
% 4 5-----6
```

```
%
```

```
% Start Node = 1
```

```
% Goal Node = 6
```

```
% -----
```

```
graph = [
```

```
0 2 4 0 0 0;
```

```
2 0 1 3 2 0;
```

```
4 1 0 0 0 3;
```

```

    0 3 0 0 2 0;
    0 2 0 2 0 2;
    0 0 3 0 2 0
];

% Heuristic values h(n) to goal node 6
h = [5 4 3 6 2 0];

start = 1;
goal = 6;

%% -----
% Greedy Best-First Search
% Evaluation function:
%  $f(n) = h(n)$ 
% -----

open = start;
visited = false(1, size(graph,1));
greedyPath = [];

while ~isempty(open)

    % Select node having minimum heuristic value
    [~, index] = min(h(open));
    current = open(index);

    % Remove selected node from OPEN
    open(index) = [];

    if visited(current)

```

```

        continue;
    end

    visited(current) = true;
    greedyPath(end+1) = current;

    % Goal test
    if current == goal
        break;
    end

    % Find neighboring nodes
    neighbors = find(graph(current,:) > 0);

    for i = 1:length(neighbors)
        node = neighbors(i);

        if ~visited(node)
            open(end+1) = node;
        end
    end
end

fprintf('Greedy Best-First Search Path: ');
fprintf('%d ', greedyPath);
fprintf('\n');

%% -----
% A* Search
% Evaluation function:

```

```
% f(n) = g(n) + h(n)
% -----
```

```
numNodes = size(graph,1);
```

```
g = inf(1, numNodes);
```

```
g(start) = 0;
```

```
open = start;
```

```
closed = false(1, numNodes);
```

```
parent = zeros(1, numNodes);
```

```
aStarPath = [];
```

```
while ~isempty(open)
```

```
    % Calculate f(n) = g(n) + h(n)
```

```
    f = g(open) + h(open);
```

```
    % Select node with minimum f(n)
```

```
    [~, index] = min(f);
```

```
    current = open(index);
```

```
    % Remove current node from OPEN
```

```
    open(index) = [];
```

```
    if closed(current)
```

```
        continue;
```

```
    end
```

```

closed(current) = true;

% Goal test
if current == goal
    break;
end

% Find neighboring nodes
neighbors = find(graph(current,:) > 0);

for i = 1:length(neighbors)

    node = neighbors(i);

    % Cost from current node to neighbor
    tentative_g = g(current) + graph(current,node);

    if tentative_g < g(node)

        % Update cost
        g(node) = tentative_g;

        % Store parent
        parent(node) = current;

        % Add node to OPEN if not already present
        if ~ismember(node, open)
            open(end+1) = node;
        end
    end
end
end

```

```
end
```

```
%% Reconstruct A* path
```

```
current = goal;
```

```
while current ~= 0
```

```
    aStarPath = [current aStarPath];
```

```
    if current == start
```

```
        break;
```

```
    end
```

```
    current = parent(current);
```

```
end
```

```
%% Display Results
```

```
fprintf('\nA* Search Path: ');
```

```
fprintf('%d ', aStarPath);
```

```
fprintf('\n');
```

```
fprintf('A* Total Cost: %d\n', g(goal));
```